NeuroGuard AI v2.4 - architecture as implemented (app.py, core/*.py, mcp_server/neuroguard_mcp_server.py)

All LLM roles: gpt-4o-mini, temperature 0, OpenAI-compatible Vocareum endpoint | contract v1.3.0 | colour: blue UI, red guard, yellow LLM agent, green deterministic code, purple MCP, cyan observability, grey data

LangGraph state machine (NODE_ROLES): PLANNER = planner | EXECUTOR = orchestrator + 4 specialists | VALIDATOR = safety reviewer + gate

Clients

Web UI: Streamlit app.py
(assess form + Chat tab: st.chat_input, conversation memory)
live on Streamlit Community Cloud

HTTP clients: REST + OpenAPI /docs

API gateway: FastAPI (api/main.py)

POST /v1/cases GET /v1/cases/{id}
GET /v1/receipts/{id}/verify POST /v1/chat
POST /v1/webhooks/case (HMAC signature)
X-API-Key auth, pydantic validation, CORS
triggers: API, webhook, cron CLI
(scripts/scheduled_batch.py)

Durable message queue (core/job_queue.py)

jobs + events tables: SQLite (WAL) default,
PostgreSQL (SKIP LOCKED) or Redis optional
leases, retries, crash recovery
A2A bus events published (topic a2a)
workers: core/worker.py

Data & memory layer

Vector store: FAISS IndexFlatIP, persisted (core/vector_store.py)
SQLite default | PostgreSQL via DATABASE_URL (core/db.py)
In-process cache | Redis via REDIS_URL (core/cache.py)

Run: Streamlit Cloud (UI) + uvicorn locally (API)
LLM = gpt-4o-mini via Vocareum

worker -> run_case()

UI / chat

Streamlit UI (app.py)

profile form, note, audience, consent,
RAG mode | lab report (image / PDF)
live progress | 13 result tabs incl.
Context-window inspector | Chat tab
follow-up agent binds
consult_evidence_researcher
(agent as a tool, core/agent_tools.py)

Input guardrails (core/guardrails.py)

PHI redaction -> prompt injection
-> BE FAST emergency (9-1-1)
-> dosing -> out of scope
+ profile validation | no LLM
multi-modal intake (partial):
core/multimodal.py vision / pypdf

Run contract (harness_contract.json)

SHA-256 hashed before first token
36 LLM | 32 tools | 110k tok | 300 s
per-agent permissions, 12 checks
context, confidence, pricing,
learning, batch policies

Planner node (LLM)

MCP episode_recall: similar past
episodes + learned hints
-> structured Plan (ordered steps,
watch-outs, rationale)
planner -> executor pattern

Executor: orchestrator (LLM)

executes the plan: next agent, skill,
instruction, reason (structured)
code overrides: eligibility, FINISH
only after review | 9 turns,
2 visits/agent, 1 revision | every route
-> decision reasoning log

CaseState (LangGraph)

short-term memory per run:
profile, plan, learning, intake,
analyst, risk, evidence, coverage,
retrieved, draft, self_critique,
review, route, trace[]

blocked -> refusal / emergency message
(+ signed receipt, 0 LLM calls)

episode_recall

Task + skill

A2A-style message bus (core/a2a.py): Agent Cards, Task -> Message -> Artifact, inproc:// endpoints (no network)

orchestrator
FINISH

Intake

skill: note-screening
MCP: screen_note, memory_read
(BE FAST signs, prior cases)
ReAct loop

Risk Analyst

skill: cohort-comparison
MCP: predict_risk, explain_factors,
cohort_stats, calculator
+ model://risk-card | ReAct

Evidence Researcher (3 RAG modes)

classical: fixed semantic_search per
topic BEFORE the LLM, docs injected
agentic (default): LLM calls
search_guidelines + graph_traverse
agentic search: files, skills, SQL,
web, memory, shell | coverage ->
re-retrieve (max 2) | context_window

Clinical Writer (reflection)

skills: risk-communication |
patient-friendly-summary
prompt clinical_report; evidence
packed to 3500 tokens; learned hints
self-critique: checks + quote-
verified LLM critique, 1 revision

Safety Reviewer (validator)

skill: red-team-review
12-check invariant verifier
(code) + LLM red team (prompt
red_team_review), quote-
confirmed | revise x1

Tool-using agents run a ReAct loop: Thought -> LLM tool_calls (bind_tools) -> pre-tool-use hook (allow/deny, clamp, pin) -> MCP Observation (PHI-redacted, truncated to 1500 tokens) -> context (compacted at 12000), max 4 rounds

load_body()

Permission engine + pre-tool-use hook

least privilege per agent (bind_tools +
re-check); arg clamps (k, limit, hops,
expr length); tool budget; session_id
and learned boost_factors pinned by
harness; memory_write needs consent;
shell argv allowlist, URL SSRF guard,
read-only SQL, own-catalog skills only
(re-checked in the server) | audit log

stdio
JSON-RPC

MCP server: official mcp SDK (FastMCP), stdio subprocess

20 tools: predict_risk, explain_factors, cohort_stats, screen_note, search_guidelines, graph_traverse,
calculator, memory_read, memory_write, episode_recall, episode_write, semantic_search,
search_files, load_skill, execute_query, web_search, web_fetch, get_memory, shell, pubmed_search
6 resources: registry://tools, guideline://index, model://risk-card, dataset://stroke-card, policy://harness-contract, graph://knowledge-graph
+ 1 resource template: guideline://{chunk_id} | 2 prompts: clinical_report, red_team_review
server-side policy engine (core/server_policy.py) re-checks every tools/call | tool registry: registry://tools
client: core/mcp_client.py + langchain-mcp-adapters; OPENAI / NG_RECEIPT / GH keys stripped from server env

write

Validator: finalize + confidence gate

re-runs the 12 checks on the final draft
confidence = weighted calibration, cohort n,
topic coverage, verifier, red team, grounding
< 0.4 refuse | < 0.7 clinician review
clinician sign-off if high risk / red flags
memory_write (consent) + episode_write (harness)

Skills (skills/, core/skills.py)

6 SKILL.md; catalog = name +
description only; full body
injected when the orchestrator
selects it (progressive disclosure)

Context windows (core/context.py)

tiktoken o200k_base counts
truncate / compact / pack evidence
window 128k, budgets per agent
slot inspector: system, skills,
tools, memory, history, user, calls, docs

Decision log + observability (core/harness.py, observability.py)

RunContext.decide(): actor, options,
chosen, rationale, evidence | one span
per LLM / MCP call -> OTLP-JSON export

Cost / token economics

usage per call x list price
USD 0.15/1M in, USD 0.6/1M out
per-agent cost; budgets cap spend

Signed receipt (core/pipeline.py)

HMAC-SHA256 over contract, server, inputs,
report, trace, decision-log hashes + audit head,
cost, confidence | clinician decision re-signs it
and is written back to episodic memory

P1 data + P2 statistics

Stroke Prediction Dataset
(Kaggle, 5,109 rows)
core/data.py cleaning log
core/stats.py: chi-square,
Welch t, odds ratios, Wilson CIs

P3 risk model (core/model.py)

LR / RF / GBM, 5-fold CV
served: calibrated LR
exact log-odds explanations
threshold: OOF recall >= 0.80
calibration curve -> confidence

P4 notes (core/notes.py)

CDC BE FAST rules
TF-IDF + LR on
MTSamples (CC0)
advisory neuro-note
probability

Hybrid RAG + vector DB (core/rag.py, vector_store.py)

30 CDC / NHLBI excerpts
sparse TF-IDF 1-2 gram (0.6)
+ dense 24-d LSA in FAISS (0.4)
+ topic / learned boost (cap 0.06)
PubMed E-utilities (pubmed_search)

Graph RAG (core/graph_rag.py)

networkx: 49 nodes (10 factor,
9 condition, 30 chunk), 144 edges
weighted multi-hop traversal
factor -> chunk, factor ->
condition <- chunk, with paths

Long-term + episodic memory

SQLite / PostgreSQL: case
summaries (consent) | episodes
-> learning loop (hints, boosts)
conversation memory: chat turns
(PHI-redacted, consent, bounded)

Scale (partial): queue + workers (above) and core/batch.py runs a cohort with asyncio.Semaphore(3) + one shared token bucket (120 LLM calls/min) awaited before every LLM call. Honest scope: A2A is in-process; public educational data; not a medical device.

Tests: pytest suite (tests/, CI on Python 3.13 + 3.14, LLM mocked) + scripts/verify_mcp.py + live end-to-end runs (pipeline and FastAPI)

Repo: https://github.com/sandipdey2/neuroguard-agentic-rag | Live demo: https://neuroguard-agentic-rag-4ywylkye6e2rgzavdb5xrb.streamlit.app/